

ЛЕКЦИЯ 10

Въведение в техниките на програмиране
на SQL



5th Edition

Elmasri / Navathe

Съдържание на лекцията

- General Constraints as Assertions
- Views in SQL
- Database Programming
- Embedded SQL
- Functions Calls, SQL/CLI
- Stored Procedures, SQL/PSM

Цели на лекцията

- Спецификация на общи **ограничения** през assertions
- SQL възможности за дефиниране на изгледи (views, virtual tables)
- Различни техники за достъп и обработка на БД с езици за програмиране
 - Java, C++ и др.

Constraints as Assertions

- Общи ограничения: ограничения, които не попадат в основните SQL категории.
- Механизъм: **CREATE ASSERTION**
 - Компонентите включват:
 - Име на ограничението,
 - Последвано от CHECK,
 - Последвано от условие
 - Командата не се поддържа в SQL Server (вместо нея има CREATE RULE), MySQL, Oracle и PostgreSQL.
 - Поддържа се от Sybase.

Assertions: пример

- “Заплатата на служителите трябва да бъде по-малка от заплатата на мениджъра на отдела, в който работи служителя.”

```
CREATE ASSERTION SALARY_CONSTRAINT  
CHECK (NOT EXISTS (SELECT *
```

```
FROM EMPLOYEE E, EMPLOYEE M,  
      DEPARTMENT D  
WHERE E.SALARY > M.SALARY AND  
      E.DNO=D.NUMBER AND  
      D.MGRSSN=M.SSN) )
```

constraint
name,
CHECK,
condition

Използване на общи Assertions

- Задава се заявка, която удовлетворява условието; включва се в `NOT EXISTS` клауза
- Резултатът от заявката трябва да бъде празно множество
 - Ако не е така, ще се наруши твърдението.

SQL Triggers

- Цел: наблюдение на БД и предприемане на действия, когато настъпи дадено условие.
- Тригерите са изрази, синтактично подобни на твърденията (assertions) и включват:
 - Събитие (Event)
 - Като операция insert, deleted или update
 - Условие (Condition)
 - Действие (Action)
 - Предприема се, когато условието се удовлетвори
- Поддържат се от SQL Server, MySQL, Oracle и PostgreSQL с модификации.

SQL Triggers - синтаксис

```
CREATE TRIGGER trigger_name  
{BEFORE | AFTER} {[DELETE] | [INSERT] | [UPDATE] [OF  
column [,...n]]}  
ON table_name  
[REFERENCING {OLD [ROW] [AS] old_name | NEW  
[ROW] [AS] new_name  
OLD TABLE [AS] old_name | NEW TABLE [AS]  
new_name}]  
[FOR EACH { ROW | STATEMENT }]  
[WHEN (conditions)]  
code block
```


SQL Triggers: пример

- Тригер за сравнение на заплатата на служител с тази на неговия пряк началник при операция insert или update:

```
CREATE TRIGGER INFORM_SUPERVISOR
BEFORE INSERT OR UPDATE OF
SALARY, SUPERVISOR_SSN ON EMPLOYEE
FOR EACH ROW
    WHEN
        (NEW.SALARY > (SELECT SALARY FROM EMPLOYEE
                        WHERE SSN=NEW.SUPERVISOR_SSN) )
    INFORM_SUPERVISOR
    (NEW.SUPERVISOR_SSN, NEW.SSN) ;
```

Views in SQL

- Изгледът е “виртуална” таблица, която се създава от други таблици.
- Позволява ограничен брой операции за промяна на данни
 - Защото таблицата може да не се съхранява физически
- Позволява пълен набор от заявки
- Удобен е при изразяване на множество операции.
- Поддържа се с модификации в SQL Server, MySQL, Oracle и PostgreSQL.

Спецификация на изгледи (1)

- SQL команда: **CREATE VIEW**
 - Име на изгледа
 - Възможен списък от имена на атрибути (напр. При аритметични операции или при необходимост от различни имена от тези на атрибутите в базовите релации)
 - Заявка, задаваща съдържанието на таблицата

CREATE VIEW view_name [(column list)]

AS

(SELECT_statement

[WITH [CASCADED | LOCAL] CHECK OPTION])

Спецификация на изгледи (2)

- Клаузата *WITH CHECK OPTION* се ползва само при изгледи, които позволяват актуализации на базовата таблица.
- Опциите *CASCADE* и *LOCAL* на клаузата *CHECK OPTION* се ползват при вложени изгледи.
 - *CASCADE* изпълнява каскадна проверка на *CHECK OPTION*.
 - *LOCAL* изпълнява проверката само за текущия изглед.

SQL Views: пример

- Задава различна WORKS_ON таблица

```
CREATE VIEW WORKS_ON_NEW AS  
SELECT FNAME, LNAME, PNAME, HOURS  
FROM EMPLOYEE, PROJECT, WORKS_ON  
WHERE SSN=ESSN AND PNO=PNUMBER  
GROUP BY PNAME;
```

Използване на виртуална таблица

- Можем да зададем SQL заявка за новосъздадена таблица (изглед) :

```
SELECT FNAME, LNAME  
      FROM WORKS_ON_NEW  
      WHERE PNAME= 'Seena' ;
```

- Когато повече не е необходим изгледа, може да се изтрие с:

```
DROP WORKS_ON_NEW;
```

Ефективна реализация на изгледи (1)

- Промяна на заявката:
 - Представя заявката на изгледа като заявка в използваните таблици от БД
- Недостатък:
 - Неефективно за изгледи, дефинирани през комплексни заявки
 - Особено, ако допълнителни заявки се прилагат към изгледа в малък период от време

Ефективна реализация на изгледи (2)

- Материализация на изгледа:
 - Изиксва физическо създаване и поддържане на временна таблица
- Допускане:
 - Може да следват други заявки
- Отнася се до:
 - Поддържане на съответствие между базовата таблица и изгледа при актуализация на базовата таблица
- Стратегия:
 - Инкрементална актуализация

Актуализация на изгледи

- Актуализацията на един изглед без сложни операции:
 - Актуализацията трябва да се трансформира в актуализация на използваната таблица
- Изгледи, изискващи join:
 - Актуализацията *може* да се трансформира в актуализация на използваните базови релации
 - Не винаги е възможно

Изгледи, които не могат да се актуализират

- Изгледи, използващи групиране и сложни функции, не могат да се актуализират.
- Изгледи, дефинирани върху множество таблици чрез join операция, не могат да се актуализират.
- **WITH CHECK OPTION**: трябва да се добави към дефиницията на изгледа, ако изгледа ще се актуализира.
 - Клаузата позволява проверка за актуализируемост и планиране на стратегията за изпълнение.

Програмиране с БД

- Цел:
 - Да се достъпи БД от приложна програма (а не от графичния интерфейс на СУБД)
- Защо?
 - Графичният интерфейс на СУБД е удобен, но недостатъчен.
 - Основната част от операциите върху БД се извършват през приложни програми (основно на web приложения)

Подходи за програмиране с БД

- Вградени команди:
 - Конадите за БД са вградени в повечето езици за програмиране.
- Библиотека от функции за работа с БД:
 - Достъпна в езика за програмиране; известна като *API*
 - *API* = Application Program Interface
- Нов full-fledged език
 - Минимизира несъответствието от импеданса

Impedance Mismatch

- Несъответствия между езика за програмиране и модела на БД:
 - Типови несъответствия, изискващи нови вградени типове във всеки език за програмиране
 - Множество срещу обработка по записи
 - Нуждаем се от специален итератор за цикъл в резултата на заявката (тя връща множество) и обработка на отделните стойности

Алгоритъм в програмирането с БД

1. Клиентската програма *отваря връзка* към сървъра на БД.
2. Клиентската програма *изпраща заявка и/или актуализации* към БД.
3. Когато достъпът до БД не е необходим повече, клиентската програма *затваря (терминира) връзката*.

Embedded SQL

- Повечето SQL оператори могат да се вграждат в езиците за програмиране с общо предназначение като COBOL, C, Java
- Един вграден SQL оператор се различава от операторите на езика за програмиране като се загражда в **EXEC SQL** или **EXEC SQL BEGIN** и съответстващия **END-EXEC** или **EXEC SQL END** (или ;)
 - Синтаксисът може да варира между различните езици
 - *Споделените променливи* (използвани и в двата езика) обикновено се префискират с : в SQL.

Пример: декларация на променлива в езика С

- Намиращите се в **DECLARE** променливи са споделени и могат да се ползват в SQL операторите (ако се предхождат от :).
- **SQLCODE** се ползва за комуникация errors/exceptions между БД и програмата

```
int loop;
```

```
EXEC SQL BEGIN DECLARE SECTION;
```

```
    varchar dname[16], fname[16], ...;
```

```
    char ssn[10], bdate[11], ...;
```

```
    int dno, dnumber, SQLCODE, ...;
```

```
EXEC SQL END DECLARE SECTION;
```


SQL команди за връзка към БД

- Връзката (възможни са множество връзки, но само една е активна в даден момент) се осъществява чрез:
**CONNECT TO server-name AS connection-name
AUTHORIZATION user-account-info;**
- Промяна на активната връзка става с:
SET CONNECTION connection-name;
- Прекратяване на връзката се извършва с:
DISCONNECT connection-name;

Вграден SQL в C

Примери (1)

```
loop = 1;
while (loop) {
    prompt ("Enter SSN: ", ssn);
    EXEC SQL
        select FNAME, LNAME, ADDRESS, SALARY
        into :fname, :lname, :address, :salary
        from EMPLOYEE where SSN == :ssn;
    if (SQLCODE == 0) printf(fname, ...);
    else printf("SSN does not exist: ", ssn);
    prompt("More SSN? (1=yes, 0=no): ", loop);
    END-EXEC
}
```

Вграден SQL в C

Примери (2)

- **cursor** (итератор) се използва за обработка на множество tuples
- Командата **FETCH** премества курсора към *следващ* tuple
- **CLOSE CURSOR** показва, че обработката на резултата от заявката е приключил.

Dynamic SQL

- Цел:
 - Създаване и изпълнение на нови (некомпилирани до този момент) SQL оператори по време на изпълнение на кода.
 - Програмата получава SQL оператори от клавиатурата по време на изпълнение на кода
 - Операция point-and-click се транслира в SQL заявка
- Динамичната актуализация е относително проста; динамичната заявка може да бъде сложна
 - Защото типа и броя на извлечените атрибути е неясен по време на компилация.

Dynamic SQL: пример

```
EXEC SQL BEGIN DECLARE SECTION;
```

```
varchar sqlupdatestring[256];
```

```
EXEC SQL END DECLARE SECTION;
```

```
...
```

```
prompt ("Enter update command:", sqlupdatestring);
```

```
EXEC SQL PREPARE sqlcommand FROM :sqlupdatestring;
```

```
EXEC SQL EXECUTE sqlcommand;
```

Embedded SQL в Java

- SQLJ: един стандарт за вграден SQL в Java
- Транслаторът SQLJ преобразува операторите на SQL в Java
 - Изпълнява се през *JDBC* interface
- Трябва да се импортират множество класове
 - Като `java.sql`

Връзка от Java към БД

- JDBC:
 - Функции за SQL връзка, извикани от програма на Java
- Една програма на Java с JDBC функции може да достъпи всяка релационна СУБД, която има JDBC driver
- JDBC позволява на програмата да се свърже към различни БД (известни като *data sources*)

Алгоритъм за достъп до БД през JDBC

1. Импортираме JDBC библиотека (**java.sql.***)
2. Зареждаме JDBC driver:
Class.forName("oracle.jdbc.driver.OracleDriver")
3. Дефинираме подходящи променливи
4. Създаваме обект за връзка (през **getConnection**)
5. Създаваме обект оператор с класа **Statement**:
 - 1. PreparedStatement 2. CallableStatement
6. Идентифицираме параметрите на оператора (означени с ?)
7. Свързваме параметрите с променливите на програмата
8. Изпълняваме SQL оператора (референсиран като обект) през JDBC's **executeQuery**
9. Обработваме резултатите на заявката (върнати от обект от тип ResultSet)
 - **ResultSet** е 2-дименсионна таблица

Embedded SQL в Java: пример

```
ssn = readEntry("Enter a SSN: ");  
try {  
    #sql{select FNAME< LNAME, ADDRESS, SALARY  
    into :fname, :lname, :address, :salary  
    from EMPLOYEE where SSN = :ssn};  
}  
catch (SQLException se) {  
    System.out.println("SSN does not exist: ",+ssn);  
    return;  
}  
System.out.println(fname + " " + lname + ... );
```

Множество Tuples в SQLJ

- SQLJ поддържа 2 типа итератори:
 - *named iterator*: свързан с резултат от заявка
 - *positional iterator*: списък от типове на атрибути в резултата от заявката
- Операцията **FETCH** извлича следващия tuple в резултата от заявката:
fetch iterator-variable into program-variable

Програмиране на БД с извикване на функции

- Вграденият SQL предоставя статично програмиране на БД
- API: динамично програмиране на БД с библиотека от функции
 - Предимство:
 - Няма нужда от предпроцесор (по гъвкав метод)
 - Недостатък:
 - SQL синтаксиса се проверява по време на работа на кода.

SQL Call Level Interface

- Част от SQL стандарта
- Предоставя лесен достъп до множество БД в една и съща програма.
- Различни библиотеки (напр. **sqlcli.h** за C) трябва да бъдат инсталирани и достъпни.
- Операторите на SQL се създават динамично и се предават като string параметри в извикването.

Компоненти на SQL/CLI

- *Environment record:*
 - Следи връзките към БД
- *Connection record:*
 - Следи наличността на необходимата информация за конкретна връзка
- *Statement record:*
 - Следи наличността на информация, необходима за един SQL оператор
- *Description record:*
 - Описва tuples

Алгоритъм за C и SQL/CLI програмиране

1. Зареждаме SQL/CLI библиотеки
2. Декларираме handle променлива за запис за представените по-горе компоненти (**SQLHSTMT**, **SQLHDBC**, **SQLHENV**, **SQLHDEC**)
3. Задаваме environment record чрез **SQLAllocHandle**
4. Задаваме connection record чрез **SQLAllocHandle**
5. Задаваме statement record чрез **SQLAllocHandle**
6. Подготвяме оператор чрез SQL/CLI функция **SQLPrepare**
7. Свързваме параметрите с променливи на програмата
8. Изпълняваме SQL оператор през **SQLExecute**
9. Свързваме колоните на заявката с променливи на C през **SQLBindCol**
10. Използваме **SQLFetch** за получаване на стойностите на колоните в променливи на C

Database Stored Procedures

- Съществуващи процедури/функции (модули), съхранени локално и изпълнени от потребителя на БД.
 - В противовес на изпълнение от клиента.
- Предимства:
 - Ако процедурата се нуждае от много приложения, може да бъде извикана от кое и да е от тях (това намалява дублиранията).
 - Изпълнението от сървъра намалява комуникационната цена.
 - Подобрява моделиращата сила на изгледите.
- Недостатък:
 - Всяка СУБД има собствен синтаксис и това прави системата по-малко преносима.

Конструкции на съхранената процедура

- Синтаксис на съхранена процедура:

```
CREATE PROCEDURE procedure-name (params)  
local-declarations  
procedure-body;
```

- Синтаксис на съхранена функция:

```
CREATE FUNCTION fun-name (params) RETURNS return-  
type  
local-declarations  
function-body;
```

- Извикване на процедура или функция:

```
CALL procedure-name/fun-name (arguments);
```


SQL Persistent Stored Modules

- SQL/PSM:
 - Част от стандарта SQL за писане на постоянно съхранен модул
- SQL + stored procedures/functions + допълнителни програмни конструкции
 - Тук се включват цикли и разклонения
 - Подобрява силата на SQL

SQL/PSM: пример

```
CREATE FUNCTION DEPT_SIZE (IN deptno INTEGER)
RETURNS VARCHAR[7]
DECLARE TOT_EMPS INTEGER;

SELECT COUNT (*) INTO TOT_EMPS
  FROM SELECT EMPLOYEE WHERE DNO = deptno;
IF TOT_EMPS > 100 THEN RETURN "HUGE"
  ELSEIF TOT_EMPS > 50 THEN RETURN "LARGE"
  ELSEIF TOT_EMPS > 30 THEN RETURN "MEDIUM"
  ELSE RETURN "SMALL"
ENDIF;
```

Преглед

- Твърденията предоставят възможност за задаване на допълнителни ограничения
- Тригерите са твърдения, които дефинират автоматично изпълними действия при настъпване на специфични условия
- Изгледите създават временни (виртуални) таблици
- Една БД може да бъде достъпвана в интерактивен режим
- По-често данните в БД се обработват през приложна програма
- Различни методи за програмиране с БД:
 - Embedded SQL
 - Dynamic SQL
 - Съхранени процедури и функции